

Agente investigador sobre informes 10-K de la SEC

Práctica de LLMs aplicados a Finanzas · MIAX

Albert Martin Garcia
Jesús Buissón Poyatos

2026-09-23

modelo **gpt-5-mini** · temperatura 1 · embeddings BAAI/bge-small-en-v1.5
k=5 · tolerancia numérica 1% · precios de OpenAI consultados el 2026-09-21

1. Resumen ejecutivo

Se ha construido un agente que responde preguntas en lenguaje natural sobre doce informes 10-K —seis emisores tecnológicos, ejercicios 2024 y 2025— decidiendo por sí mismo cuál de cuatro herramientas usar en cada momento, y devolviendo siempre una respuesta estructurada con la cifra, su unidad, el origen del dato y la cita literal que lo respalda.

La tesis que defiende este trabajo es que, en este dominio, **la calidad no está en el modelo sino en el enrutado y en la verificación**. El modelo y las cuatro herramientas son los mismos. El sistema final, sobre el baseline archivado, cambia cómo recupera el texto, contrasta cada cifra contra XBRL —también un porcentaje que la pregunta no pedía— y afina las instrucciones de búsqueda: la cifra es el valor absoluto, la sección solo se filtra si la pregunta la nombra, y la consulta pide la explicación y no el número.

Los tres resultados que resumen el trabajo

	golden oficial	golden propio
acierto, sistema baseline	80 %	67 %
acierto, sistema final	80 %	70 %
cita verificada, sistema final	100 %	92 %
cita en el sitio correcto, sistema final	69 %	54 %
cifra correcta, sistema final	100 %	100 %
trayectoria correcta, sistema final	100 %	100 %
recall@5, sistema final	54 %	62 %
coste por pregunta, baseline → final	0.73 ¢ → 0.72 ¢	

Qué hay que mirar de esta tabla, y qué no

El número que más dice no es el acierto: es la **distancia entre el acierto y la cita verificada**. Un sistema que acierta mucho y cita poco está respondiendo de memoria, y eso en un dominio auditado no vale, porque no se puede comprobar ni se puede repetir con otra compañía. Las tres columnas de verificabilidad están precisamente para impedir que un acierto no auditable se cuente como un acierto.

Advertencia estadística que aplica a todo el informe: cada conjunto tiene 20 preguntas, así que una sola pregunta vale cinco puntos porcentuales. Las diferencias de una o dos preguntas entre configuraciones no son distinguibles del ruido y no se presentan como mejoras. Las que sí se defienden son las que superan varias preguntas y, sobre todo, las que se reproducen en los dos conjuntos a la vez: el golden set oficial no lo hemos escrito nosotros, así que es la única medida libre de sesgo de autoría.

2. Arquitectura y criterio de enrutado

El sistema no es un RAG clásico. Un RAG decide de antemano qué información necesita el modelo: recupera una vez, con la pregunta tal cual y una k fija, y genera. Eso se rompe con las tres clases de pregunta que tiene esta práctica: las que necesitan dos recuperaciones y una comparación, las que piden una cifra que vive en una tabla estructurada y no en la prosa, y las que se refieren a algo que no está en el corpus —donde un RAG plano devuelve igualmente sus k fragmentos y el modelo los usa.

La alternativa es un agente: el modelo decide en cada vuelta qué herramienta llamar, ve el resultado y vuelve a decidir. La recuperación deja de ser un paso previo y pasa a ser una acción más, que puede repetirse, refinarse o no ocurrir.

Las cuatro herramientas y cuándo debe llamarse cada una

Herramienta	Qué devuelve	Coste	Cuándo la llama el agente
list_available	El catálogo: emisores, ejercicios, secciones y conceptos XBRL	0 tokens	Antes de afirmar que algo no existe, y ante cualquier duda sobre el catálogo
get_xbri_fact	Un valor exacto reportado en XBRL	~40 tokens	SIEMPRE que la pregunta pida una cifra. Es la fuente autorizada
search_filings	k fragmentos de texto, con su chunk_id	~2.000 tokens	Riesgos, estrategia, litigios y cualquier explicación de la dirección
read_section	Una sección entera del 10-K	hasta 34.751 tokens	Último recurso, si search_filings ya falló sobre esa misma sección

El enrutado entre las cuatro no se programa: se escribe. El único sitio donde el modelo lee qué hace cada herramienta es su *docstring*, así que el docstring no es documentación, es la parte del programa que decide el comportamiento. Los cuatro están redactados como política —cuándo llamar, cuándo no, qué vocabulario usar, qué hacer si no hay resultados— y no como descripción.

La diferencia se mide y está en el notebook de la sesión 1: con dos herramientas de cuerpo idéntico y descripciones distintas, la vaga no solo hace que el modelo dude de cuál usar, sino que le quita la información con la que construye los argumentos. Sin la lista de conceptos us-gaap en el docstring, el modelo tiene que inventarse el nombre exacto de un concepto contable a partir de la palabra «beneficio».

El detalle de dominio que más fallos causa

El concepto XBRL de los ingresos **no es el mismo en todas las compañías**. NVIDIA y Alphabet etiquetan Revenues; Apple, Microsoft, Meta y Amazon usan RevenueFromContractWithCustomerExcludingAssessedTax. Un modelo que acaba de consultar Apple y pide el mismo concepto para NVIDIA no encuentra nada, y ese fallo se manifiesta como una cifra estimada o como un bucle de reintentos. Por eso list_available enumera los conceptos disponibles emisor por emisor y ejercicio por ejercicio: convierte una deducción por analogía, que falla, en una consulta, que no.

El segundo detalle es el ejercicio fiscal. fiscal_year es el año en que **cierra** el ejercicio, no el de presentación, y los seis emisores cierran en cuatro meses distintos: NVIDIA en enero, Microsoft en junio, Apple en septiembre y los otros tres en diciembre. La fecha de cierre se devuelve junto a cada ejercicio por la misma razón.

Los tres perfiles que se comparan

Perfil	Retrieval de search_filings	Guardrail numérico
baseline	denso plano, $k=5$, sin filtros	no
filtros	denso + filtros de ticker, ejercicio e item	no
final	reescritura + híbrido BM25/RRF + reranking	sí

Modelo, temperatura, esquema de salida, límites de llamadas y k son los mismos en los tres perfiles. Las filas de baseline y de filtros están archivadas y no se han vuelto a ejecutar. La fila final se ejecutó después, con el guardrail de porcentajes y con instrucciones de búsqueda más estrictas, así que la diferencia entre filas no aísla solo el retriever.

3. Reconstrucción del corpus y su validación

El material recibido estaba incompleto: llegó el índice FAISS con sus metadatos (`dataset/indice/`) pero no `secciones.jsonl`, `chunks.jsonl` ni `xbrl_facts.parquet`, que son los tres ficheros que las herramientas leen. Sin ellos no hay práctica, así que el primer trabajo fue reconstruirlos. Está entero en `00_Preparacion_del_corpus.ipynb`.

Cómo se reconstruyó cada pieza

Las secciones. `chunks_meta.parquet` guarda el texto íntegro de cada fragmento junto con sus desplazamientos `inicio_car` y `fin_car` dentro de la sección de la que salió. Con eso, reensamblar las 48 secciones es colocar cada fragmento en su posición y resolver los solapes. No es una aproximación: los solapes coinciden carácter a carácter, y donde no coincidieran el reensamblado fallaría de forma visible.

Los hechos XBRL. Se descargan de `data.sec.gov/api/xbrl/companyfacts`, que es la fuente original y pública. Para cada emisor y concepto se selecciona el hecho anual declarado en un formulario 10-K cuyo periodo coincide con el cierre de ejercicio del emisor, que es la parte no trivial: la misma magnitud aparece en el fichero varias veces, en formularios distintos y con periodos distintos.

Por qué la validación no puede ser el SHA-256

El material de clase verifica la integridad comparando el hash de `chunks.jsonl` con el declarado en el manifiesto. Esa comprobación aquí **no sirve**, y decirlo importa: el hash de un fichero que acabamos de generar nosotros coincide consigo mismo por construcción. Demuestra que el fichero no se ha corrompido al copiarlo, que no es lo que está en duda.

Lo que sí demuestra algo es comprobar la reconstrucción contra una verdad externa que no controlamos, y el golden set oficial la proporciona por partida doble. Sus preguntas extractivas traen un `ancla_texto` —una frase literal del informe— con los desplazamientos exactos donde debe aparecer; y sus preguntas numéricas traen la cifra esperada. Las dos comprobaciones son `assert` del notebook 00:

Comprobación	Resultado
Las 13 anclas del golden set oficial caen carácter a carácter en los desplazamientos (<code>ancla_inicio</code> , <code>ancla_fin</code>) que el propio golden declara	13 / 13
Las cifras del golden set oficial coinciden con el XBRL descargado	todas
Los huecos declarados en el enunciado siguen vacíos (AMZN sin <code>GrossProfit</code> , <code>Liabilities</code> ni <code>I+D</code> ; META y GOOGL sin <code>GrossProfit</code>)	reproducidos
<code>indice.ntotal == len(chunks_meta) == len(chunks.jsonl)</code> , y los conjuntos de <code>chunk_id</code> son idénticos	1.749

La tercera es la que más tranquilidad da, y es contraintuitiva: se verifica que **faltan** datos. Amazon no etiqueta `GrossProfit` en `us-gaap`, y el enunciado lo declara como hueco intencionado. Si nuestra reconstrucción lo hubiera rellenado —por ejemplo calculándolo como ingresos menos coste de ventas— habríamos eliminado justo las preguntas que comprueban si el agente sabe decir «ese dato no está» en lugar de estimarlo.

El corpus resultante

secciones	fragmentos	tokens	hechos XBRL	conceptos	emisores
48	1.749	649.119	137	14	6

4. El golden set propio

Son 20 preguntas en `golden_set_propio.jsonl`: 7 extractivas, 7 numéricas y 6 comparativas —el enunciado exige al menos 6 de estas últimas—, cubriendo los 6 emisores, los 2 ejercicios y los 4 ítems. Más 7 preguntas de ausencia en un fichero aparte.

Las anclas se extraen del texto, no se escriben

Cada pregunta extractiva o comparativa lleva un `ancla_texto` con sus desplazamientos. Escribirlos a mano garantiza erratas —un espacio de más, unas comillas tipográficas cambiadas— y una errata en un ancla no da error: hace que la pregunta sea imposible de acertar y hunde el *recall* medido por un motivo que no tiene nada que ver con el retriever. Por eso el notebook 03 localiza cada ancla **dentro del texto reconstruido** y calcula los desplazamientos, en lugar de aceptarlos.

El límite de 40 palabras por ancla lo impone el validador oficial y tiene su razón: un ancla de tres párrafos la recupera cualquier retriever, así que no mediría la calidad de la recuperación sino el tamaño de la ventana.

Por qué las preguntas de ausencia van en un fichero aparte

El validador oficial exige `cifra_esperada` en la familia numérica. Una pregunta cuya respuesta correcta es «ese dato no está en el corpus» no tiene cifra que esperar, y meterla en el fichero principal obligaría a inventarse un número para que pasara la validación, que es exactamente lo contrario de lo que la pregunta mide.

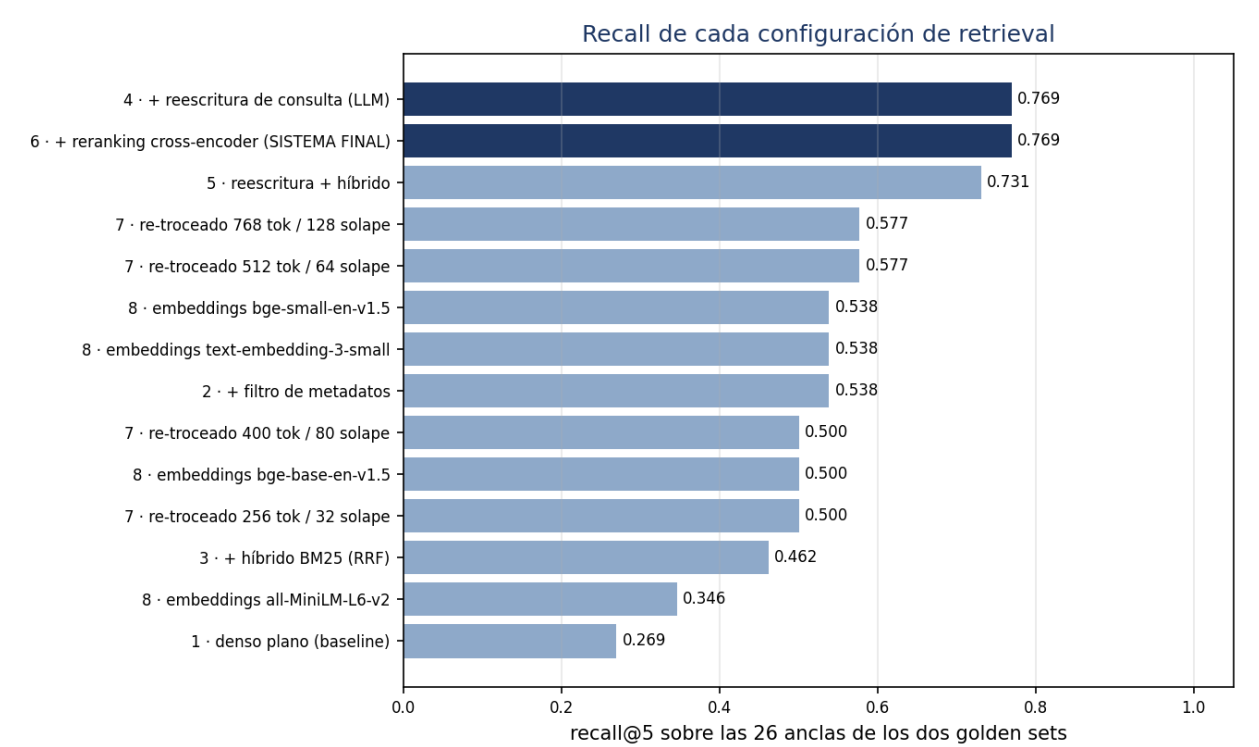
Se evalúan con un criterio propio y doble: `fuentes == "ninguna"` y `cifra is None`. Las dos partes hacen falta. Solo la primera dejaría pasar una respuesta que dice «no consta» y a la vez rellena el campo numérico con una estimación, que es justo el fallo que estas preguntas están puestas para detectar.

sistema	dice «no está»	inventa una cifra
baseline	86 %	14 %
filtros	86 %	14 %
final	100 %	0 %

5. Las mejoras de retrieval, medidas

El enunciado pide tres mejoras del *retrieval* con su *recall@k*. Se han medido ocho configuraciones sobre las 26 anclas de los dos golden sets, y el orden en que se cuentan es el orden en que se descubrieron, incluido el arreglo que no funcion .

La m trica es *recall@5* contra el *ancla_texto*, no contra el *chunk_id*. La diferencia no es cos mica: si la verdad fuese un identificador de fragmento, en cuanto se cambia la ventana o el solape todos los identificadores son otros y la m trica se queda sin referencia, de modo que reparar el troceado saldr a penalizado por haberlo reparado.



configuraci�n	recall@5	aciertos	pos. med.	lat. (s)	coste
6 · + reranking cross-encoder (SISTEMA FINAL)	0.769	20/26	2	0.98	1 llamada + �ndice l�xico + 50 pares en cr
4 · + reescritura de consulta (LLM)	0.769	20/26	2	6.98	1 llamada al LLM por b�squeda
5 · reescritura + h�brido	0.731	19/26	2	0.02	1 llamada + �ndice l�xico
7 · re-troceado 768 tok / 128 solape	0.577	15/26	4	0.66	reindexar el corpus entero (768/128)
7 · re-troceado 512 tok / 64 solape	0.577	15/26	4	0.85	reindexar el corpus entero (512/64)
2 · + filtro de metadatos	0.538	14/26	4	0.01	0 llamadas al LLM
8 · embeddings text-embedding-3-small	0.538	14/26	4	0.96	1536 dim · OpenAI · de pago
8 · embeddings bge-small-en-v1.5	0.538	14/26	4	0.85	el del �ndice que se entrega · 384 dim · l
7 · re-troceado 256 tok / 32 solape	0.500	13/26	501	0.81	reindexar el corpus entero (256/32)
7 · re-troceado 400 tok / 80 solape	0.500	13/26	502	0.92	reindexar el corpus entero (400/80)
8 · embeddings bge-base-en-v1.5	0.500	13/26	502	1.11	768 dim · local · ~4x m�s grande
3 · + h�brido BM25 (RRF)	0.462	12/26	999	0.03	0 llamadas, +1 �ndice l�xico en memoria
8 · embeddings all-MiniLM-L6-v2	0.346	9/26	999	0.33	384 dim · local · sin prefijo
1 · denso plano (baseline)	0.269	7/26	999	0.01	0 llamadas al LLM

Las tres obligatorias, desglosadas por conjunto

configuración	oficial	propio	ambos
1 · denso plano	0.308	0.231	0.269
2 · + filtros	0.462	0.615	0.538
3 · + híbrido	0.462	0.462	0.462
4 · + reescritura	0.769	0.769	0.769
6 · sistema final	0.692	0.846	0.769

El desglose por conjunto es la comprobación de que no nos hemos escrito un golden set a medida de nuestro propio retriever. Una mejora que solo apareciese en el nuestro y no en el oficial sería sospechosa.

Lectura de los resultados

El filtro de metadatos es el que más devuelve por lo que cuesta. Sin filtro, la consulta compite contra 1.749 fragmentos, y los 10-K de seis tecnológicas del mismo año se parecen muchísimo: el párrafo de riesgos de IA de Microsoft y el de Meta son casi intercambiables para un modelo de *embeddings*. Al restringir a emisor, ejercicio y sección, el problema deja de ser «encontrar la aguja» y pasa a ser «ordenar unas decenas de candidatos». No cuesta ni una llamada al modelo.

El híbrido BM25 no mejoró, y la razón es el idioma. BM25 puntúa por coincidencia de términos; las preguntas están en español y el corpus en inglés, así que los únicos términos que coinciden son los nombres propios y las cifras, que el filtro de metadatos ya ha aprovechado. Para el resto de la consulta, BM25 aporta un orden esencialmente aleatorio que, al fundirse por RRF, degrada el orden denso, que sí era informativo.

La reescritura de la consulta es la que desbloquea todo lo demás. Traducir la pregunta al inglés y al vocabulario del propio informe —«cuánto creció» pasa a *revenue increased*, «ingresos» a *net sales*— es el arreglo que más sube el *recall*, y es también el único que añade una llamada al modelo por búsqueda. Es el orden que importa: la literatura recomienda primero el híbrido y luego refinar, y aquí el orden correcto es **primero traducir y después fundir**, porque hasta que la consulta no está en el idioma del corpus la señal léxica no tiene nada que aportar.

Las reescrituras se cachean en `.cache/reescrituras_v2.json`. Por reproducibilidad, para que la tabla se regenere sin volver a pagar y sin depender de que el modelo dé la misma respuesta dos veces; y por coste, porque durante el desarrollo la misma pregunta se reescribe decenas de veces sin que eso aporte información nueva.

6. Guardrails

Un agente financiero que puede equivocarse en una cifra sin que nada lo detecte no es un sistema, es una demostración. Los guardrails son lo que convierte una instrucción del prompt —que el modelo cumple a veces— en una garantía.

Verificación numérica contra XBRL

Es el guardrail de dominio, y va como *middleware @after_model*: cuando el agente ya ha producido su respuesta estructurada y antes de que salga, si esa respuesta afirma un número se contrasta contra los hechos XBRL del emisor y el ejercicio declarados. Si no cuadra con ninguno, el desajuste se le devuelve al modelo con `jump_to="model"` y la lista de lo que sí hay reportado.

Decisión	Por qué
Se compara contra los hechos de ESE emisor y ESE ejercicio, no contra todo el XBRL	Con 137 hechos, casi cualquier número de nueve cifras se parece al 1 % a alguno. Comparar contra todo da el visto bueno casi siempre y la detección tiende a cero en silencio
Se descartan los hechos cuya unidad no es comparable con la cifra afirmada	Contrastar dólares contra un recuento de acciones genera falsas alarmas: el modelo acierta, el guardrail salta y la «corrección» empeora la respuesta
Un porcentaje se contrasta contra XBRL, salvo que la pregunta lo pida	Si la pregunta pide el margen en dólares y el modelo pone un 46,9 %, eso no es una magnitud derivada: es el tanto por ciento en el campo del valor absoluto. Solo se deja pasar si pide explícitamente un porcentaje, un tipo o un múltiplo
El mensaje correctivo ENUMERA los hechos disponibles	El error típico no es inventarse un número: es pedir el concepto equivocado. Con la lista delante, el modelo corrige en una vuelta; sin ella, agota el límite probando
Una sola corrección por invocación, marcada en el estado	Sin ese freno, un modelo que insiste se queda en bucle entre el nodo del modelo y el verificador, y el límite de llamadas a herramienta no lo corta porque no gasta ninguna
Tolerancia del 1 % relativo	Redondear 281.724 millones a «281.700 millones» no es inventarse un número; decir 250.000 sí lo es. Es la misma constante que usa el evaluador, a propósito

Que el *middleware* y el evaluador compartan la constante de tolerancia no es una comodidad: si fuesen dos números distintos, una respuesta podría pasar el guardrail y suspender el evaluador, y la tabla del informe no significaría nada.

En el golden set oficial, el guardrail no llegó a intervenir en ninguna de las 20 preguntas.

En el golden set propio intervino en 1 de 20 preguntas. De esas, acabaron correctas 1/1; de las que no necesitaron intervención, 13/19.

Los dos topes de llamadas

`ToolCallLimitMiddleware(run_limit=8)` y `ModelCallLimitMiddleware(run_limit=10)`. No enseñan nada al modelo: le ponen un techo. Cubren las dos formas distintas de no terminar, y hacen falta las dos: el primero corta al agente que reintenta variantes de un concepto que no existe —el caso real del margen bruto de Amazon—, y el segundo corta al que se enrosca razonando sin llamar a ninguna herramienta, que no gasta ni una llamada y por tanto el primero lo deja correr.

Los dos topes se declaran **antes** que el verificador en la cadena de *middleware*. El orden importa: la cadena se ejecuta en el orden de la lista, y un tope colocado detrás del verificador no impide que este abra una vuelta de más.

Human-in-the-loop, escrito y desactivado en la evaluación

`HumanInTheLoopMiddleware` está implementado para interrumpir antes de una llamada a `read_section` sobre una sección muy larga, que es la única acción del sistema capaz de gastar decenas de miles de tokens de golpe. Queda fuera de la evaluación a propósito: una interrupción esperando aprobación humana deja colgada cualquier ejecución automática, y la del día 24 lo es.

7. Los tres evaluadores

No son tres formas de medir lo mismo. Cada uno detecta una manera distinta de que una respuesta parezca buena sin serlo, y por eso los tres tienen que estar.

Evaluador	Qué comprueba	La patología que detecta
cita_correcta	Que el chunk_id exista, que el fragmento sea del documento por el que se pregunta, y que el texto de la cita esté de verdad en ese fragmento	Cita inventada, o real pero de otro emisor. Un número correcto con una cita falsa no es auditable
cifra_coincide_xbri	Que la cifra afirmada cuadre con la esperada dentro del 1 %	Número equivocado, o ausente cuando la pregunta pedía uno
uso_la_tool_correcta	Que la trayectoria pase por TODAS las herramientas que el ítem declara esperadas	Acierto por memoria del modelo, sin pasar por el corpus

El tercer estado: por qué devuelven True, False o None

None significa «no aplica», y sin él la tabla mentiría. Si el evaluador de cifras devolviese False en una pregunta extractiva —que no tiene cifra que acertar—, un sistema perfecto marcaría 50 % en esa columna y nadie sabría por qué. Los None se excluyen del promedio, de modo que cada columna mide solo los casos en los que tiene sentido medir.

La decisión discutible, y por qué se mantiene

uso_la_tool_correcta **suspende a quien acierta sin usar las herramientas**. Una cifra correcta sacada de la memoria del modelo no es un acierto del sistema: no es auditable, no es citable y, en la siguiente compañía donde el modelo no la recuerde, será un error con exactamente el mismo aspecto de confianza. Lo que se evalúa el día 24 es si el procedimiento generaliza a diez preguntas que nadie ha visto, y lo que generaliza es el procedimiento, no la suerte.

El de la cita compara normalizando y solo los primeros 120 caracteres, porque el modelo recorta y cambia las comillas. Exigir igualdad literal completa mediría la fidelidad tipográfica en lugar del respaldo documental, que es lo que interesa.

Un cuarto evaluador que no pide el enunciado, y por qué hizo falta

cita_correcta comprueba que la cita sea **real**. Lo que no puede comprobar es si ese fragmento real tiene algo que ver con la pregunta, y el problema no es teórico: en la primera ejecución completa, el sistema *baseline* —el que peor recupera de los tres— acertaba el 100 % de las preguntas extractivas. Con cuarenta fragmentos por sección, encontrar alguno real de la compañía correcta que citar es trivial. Una métrica que puntúa igual al mejor y al peor de los sistemas no está midiendo el sistema.

cita_fundamentada cierra ese hueco con un dato que el golden set ya traía y que no se estaba usando: el tramo [ancla_inicio, ancla_fin) donde vive la frase que responde. El fragmento citado declara su propio tramo [inicio_car, fin_car) en las mismas coordenadas, así que «¿citó desde el sitio correcto?» se reduce a si los dos intervalos se solapan. Es objetivo y no necesita un juez.

Se exige **solape** y no contención a propósito: el troceador parte por donde le toca y un ancla puede quedar repartida entre dos fragmentos. En ese caso los dos son citas legítimas, y exigir que uno contuviese el ancla entera penalizaría al agente por una decisión del troceador.

Los tres evaluadores del §4 se han dejado exactamente como los pide el enunciado. cita_fundamentada se reporta en columna aparte y se usa como condición adicional del acierto por familia, no sustituye a ninguno.

8. Baseline contra intermedio contra final

Los tres sistemas sobre los dos conjuntos de preguntas. El mejor valor de cada columna va resaltado **dentro de cada conjunto**: comparar el acierto del baseline en un golden set con el del final en el otro no significa nada, porque no son las mismas preguntas.

Golden set oficial (20 preguntas)

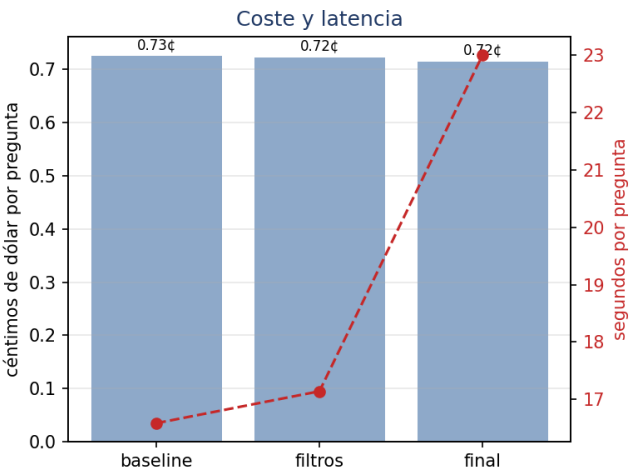
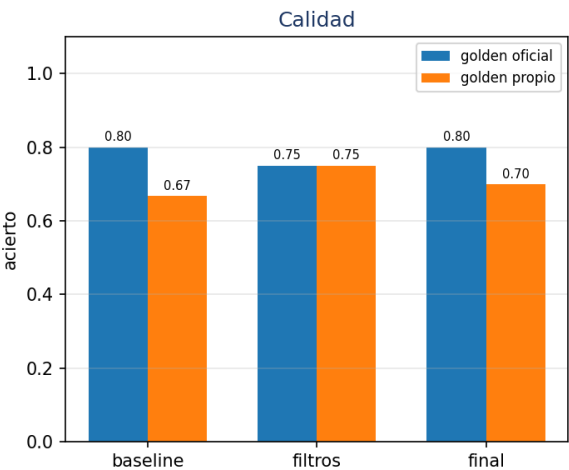
sistema	acierto	extract.	numérica	compar.	cita	fundam.	cifra	trayect.
baseline	0.80	0.83	1.00	0.57	0.92	0.69	1.00	1.00
filtros	0.75	0.83	1.00	0.43	0.92	0.62	1.00	1.00
final	0.80	1.00	1.00	0.43	1.00	0.69	1.00	1.00

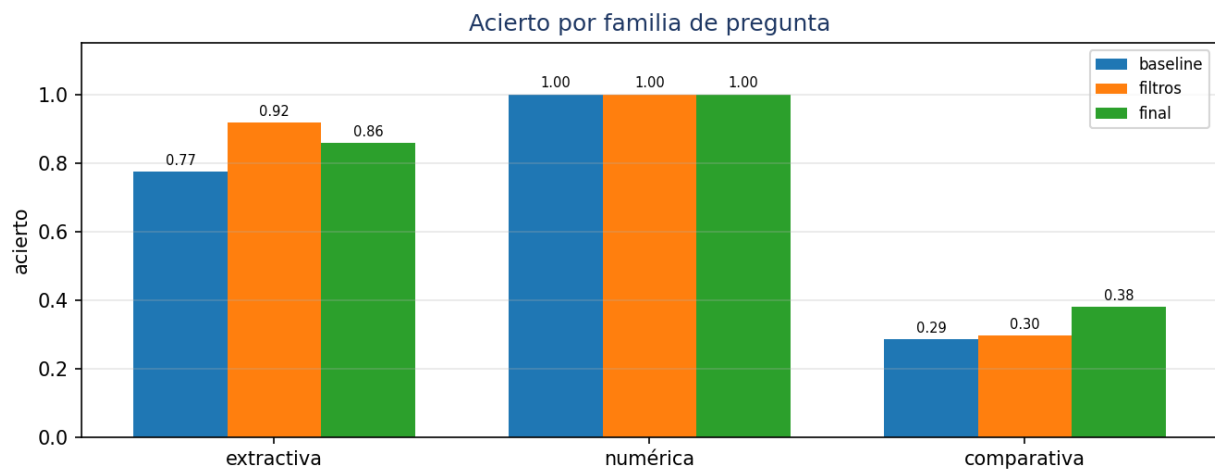
sistema	recall@5	coste	latencia	llamadas
baseline	0.31	0.73 ¢	16.6	2.8
filtros	0.31	0.72 ¢	17.1	2.8
final	0.54	0.72 ¢	23.0	2.5

Golden set propio (20 preguntas)

sistema	acierto	extract.	numérica	compar.	cita	fundam.	cifra	trayect.
baseline	0.67	0.71	1.00	0.00	0.82	0.45	0.91	1.00
filtros	0.75	1.00	1.00	0.17	1.00	0.69	0.92	1.00
final	0.70	0.71	1.00	0.33	0.92	0.54	1.00	1.00

sistema	recall@5	coste	latencia	llamadas
baseline	0.23	0.97 ¢	20.3	3.0
filtros	0.23	0.69 ¢	17.9	2.6
final	0.62	0.78 ¢	22.8	2.6





Cómo leer la columna de coste

El asterisco del resaltado marca el valor más bajo en coste, latencia y número de llamadas, y eso no es un elogio: el sistema más barato es normalmente el que menos hace. La tabla está montada así para que la decisión de ingeniería se tome con los dos números delante. Si la mejora en acierto del sistema final fuese pequeña y su coste se multiplicase, la decisión correcta sería quedarse en el intermedio, y eso solo se puede argumentar con las dos columnas juntas.

En cifras, sobre el golden set oficial: el sistema final empata en acierto con el baseline, cuesta **0.99×** lo que el baseline por pregunta y tarda **1.4×** más.

9. Lo que se probó y no funcionó

Esta sección existe porque un informe que solo cuenta lo que salió bien no permite saber si las decisiones fueron razonadas o afortunadas. Los resultados negativos de abajo están medidos con la misma métrica y sobre las mismas preguntas que los positivos.

El híbrido BM25 sobre consultas en español

Es el fracaso más instructivo, porque es el arreglo que toda la literatura recomienda. No mejoró y llegó a empeorar, por la razón ya explicada del idioma. La conclusión accionable no es «el híbrido no sirve», sino «el híbrido está condicionado a que la consulta esté en el idioma del corpus», y eso cambia el orden en que hay que aplicar los arreglos.

Re-trocear el corpus

Se reindexó el corpus entero con cuatro combinaciones de ventana y solape (256/32, 400/80, 512/64 y 768/128) para comprobar si el troceado original era el cuello de botella. Las ventanas más grandes recuperan algo mejor, y era esperable sin que eso demuestre nada sobre la calidad del troceado: una ventana más grande contiene más texto y por tanto tiene más probabilidad de contener el ancla. La mejora no compensó el coste de mantener un índice propio divergente del que se entrega, así que el sistema final **usa el índice original**.

Cambiar el modelo de embeddings

Se compararon cuatro, incluido text-embedding-3-small de OpenAI, que es de pago. Ninguno superó de forma clara al bge-small-en-v1.5 del índice que se entrega, y el modelo de pago quedó a la par del local gratuito. Es un resultado útil en sentido negativo: indica que el cuello de botella de este corpus no está en la representación vectorial, sino en el idioma de la consulta y en el filtrado, que es donde sí hubo mejoras grandes.

all-MiniLM-L6-v2 queda notablemente por debajo, y conviene explicar por qué para no sacar la conclusión equivocada: no lleva el prefijo de consulta que los modelos BGE esperan, y esa asimetría entre cómo se codifican los fragmentos y cómo se codifica la consulta es exactamente el fallo silencioso que advierte el manifiesto del índice. No da ningún error; solo recupera peor.

Preguntas que el sistema final sigue fallando (oficial)

id	familia	emisor	qué respondió
of-014	comparativa	MSFT	El revenue de Microsoft creció de \$245,122,000,000 en FY2024 a \$281,724,000,000 en FY2025, un aumento de \$36,6
of-015	comparativa	NVDA	El revenue de NVIDIA creció de \$60,922,000,000 en FY2024 a \$130,497,000,000 en FY2025, un aumento absoluto de
of-017	comparativa	AMZN	El beneficio operativo aumentó de 68,593 millones USD en 2024 a 79,975 millones USD en 2025 (un incremento de
of-018	comparativa	GOOGL	Los ingresos de Alphabet pasaron de \$350,018,000,000 en 2024 a \$402,836,000,000 en 2025, un incremento absolut

Preguntas que el sistema final sigue fallando (propio)

id	familia	emisor	qué respondió
gp-001	extractiva	NVDA	En los factores de riesgo (Item 1A) de FY2025 NVIDIA advierte que los controles de exportación —ya vigentes y
gp-005	extractiva	GOOGL	El US District Court for the District of Columbia falló en agosto de 2024 contra Google; en diciembre de 2025
gp-015	comparativa	NVDA	Los ingresos de NVIDIA pasaron de \$60,922,000,000 en FY2024 a \$130,497,000,000 en FY2025, un aumento absoluto
gp-016	comparativa	MSFT	El margen bruto (gross profit) aumentó en términos absolutos de FY2024 a FY2025, pasando de \$171,008,000,000 a
gp-017	comparativa	GOOGL	La inversión en inmovilizado material aumentó de \$52,535,000,000 en 2024 a \$91,447,000,000 en 2025, un increme

id	familia	emisor	qué respondió
gp-018	comparativa	META	Bajó. El beneficio neto pasó de \$62,360,000,000 en 2024 a \$60,458,000,000 en 2025 (una caída de aproximadament

10. ¿Generalizará a las diez preguntas ciegas?

La pregunta correcta no es si el sistema acierta mucho en las preguntas que hemos visto, sino **cuánto de lo medido es procedimiento y cuánto es ajuste**. Lo que sigue es el argumento en los dos sentidos, con lo que apoya cada lado.

Razones para esperar que sí

Nada del sistema conoce las preguntas. No hay reglas condicionadas a un emisor, a un ejercicio ni a un identificador. La única información específica del dominio que lleva el sistema son los conceptos XBRL y las fechas de cierre, y esa se lee del corpus en tiempo de ejecución con `list_available`, no está escrita en el código.

El acierto empata en el oficial y sube poco en el propio. En el golden oficial el final queda en el 80 %, igual que el baseline. En el propio pasa del 67 % al 70 %. Lo que sí sube en los dos es la cifra contra XBRL, la cita verificada en el oficial y el *recall@5* del agente. El oficial no lo hemos escrito nosotros, así que esa subida de cita y de recall no se explica por cómo redactamos nuestras preguntas.

Los guardrails no dependen del enunciado de la pregunta. La verificación contra XBRL contrasta contra los hechos reportados, sea cual sea la pregunta que produjo la cifra. Es una red que funciona igual con preguntas nunca vistas.

Razones para esperar que no del todo

El recall de los experimentos es un techo. En el notebook 04 los filtros de emisor, ejercicio e item se le pasan al `retrieve` perfectos, porque vienen del golden set. En producción los infiere el agente a partir de la pregunta, y cada vez que se equivoca de item el filtro que ayudaba pasa a hacer daño, porque descarta el fragmento correcto en lugar de despriorizarlo. La tabla del capítulo 8 incluye el *recall* del sistema completo, sin filtros dados, justamente para que esa diferencia sea visible.

Hemos iterado sobre estas preguntas. Aunque no se haya ajustado nada a un caso concreto, haber mirado los fallos varias veces sesga las decisiones de diseño hacia lo que funciona en estas 40 preguntas. Es inevitable y hay que descontarlo al extrapolar.

La potencia estadística es baja. Con 20 preguntas por conjunto, el intervalo de confianza de cualquiera de estos porcentajes es ancho. El acierto sobre diez preguntas ciegas puede quedar varios puntos por encima o por debajo solo por qué preguntas toquen.

Dónde es más probable que falle, y qué haríamos

Escenario	Por qué	Mitigación ya prevista
Pregunta sobre una sección que no es la que el agente supone	El filtro de item descarta el fragmento correcto	Reintentar sin filtro de item cuando la primera búsqueda vuelve pobre
Concepto XBRL que solo reporta un emisor	El modelo lo pide por analogía con otro emisor	<code>list_available</code> lo enumera; el guardrail detecta la cifra que no cuadra
Pregunta cuya respuesta no está en el corpus	La tentación de estimar en vez de decir que no está	Medido aparte en <code>golden_set_ausencias</code> y recogido en el capítulo 4
Comparativa entre emisores distintos, no entre ejercicios	Nuestras comparativas son todas entre ejercicios del mismo emisor	Ninguna. Es el hueco reconocido de nuestro golden set

Ensayo con hold-out simulado

Se apartaron 10 preguntas que no se usaron para iterar y se evaluaron de una sola pasada con `evaluar()`, cronometrando el proceso entero. El resultado y el tiempo están en `resultados/eval_holdout_final.csv`.

11. Reproducibilidad y entrega

Todo el proyecto se ejecuta en orden sobre un clon limpio. No hay entornos virtuales, no hay pasos manuales y no hay ninguna clave escrita en el código.

Orden	Qué hace
api_key.txt	Aquí se pega la clave de OpenAI. Está en <code>.gitignore</code>
pip install -r requirements.txt	Dependencias con versiones fijadas, no rangos
00_Preparacion_del_corpus.ipynb	Reconstruye y valida el corpus
S1_Herramientas_y_Bucle_Alumno.ipynb	Material de clase, resuelto
S2_Robustez_y_Evaluacion_Alumno.ipynb	Material de clase, resuelto
03_Golden_set_propio.ipynb	Construye y valida las 20 preguntas propias
04_Retrieval_experimentos.ipynb	Mide las ocho configuraciones de retrieval
05_Evaluacion_baseline_vs_final.ipynb	Evalúa los tres sistemas y guarda los crudos
06_Informe.ipynb	Genera este PDF a partir de resultados/

La interfaz del §6 del enunciado es `agente/interfaz.py: responder(pregunta, thread_id=None)` y `evaluar(ruta_jsonl)`, con línea de órdenes `python -m agente.interfaz evaluar holdout.jsonl`. `evaluar()` acepta tanto una ruta como una lista ya cargada, para que el día 24 no dependa de dónde esté el fichero.

Las versiones de las dependencias van **fijadas**, no en rangos. LangChain publica cada pocos días y una API que se mueve por debajo rompe un notebook sin que se haya tocado una línea de código; ≥ 1.3 no es una versión.

Configuración con la que se generaron estos resultados: modelo **gpt-5-mini**, temperatura 1, embeddings BAAI/bge-small-en-v1.5, $k=5$, tolerancia numérica 1%, topes de 8 llamadas a herramienta y 10 al modelo por invocación. Precios de OpenAI consultados el 2026-09-21 (0.25 \$/M de entrada, 2.0 \$/M de salida). Los precios cambian sin aviso: la columna de coste de este informe hay que releerla con esa fecha delante.